

جلسه اول

داده ساختارها و الگوریتم‌ها

بخش عملی | مقدمات و مبانی حل مسئله



طرح درس کلاس حل تمرین عملی

مقدمات و مبانی حل مسئله

هفته ۱

حل مسائل آرایه و رشته

هفته ۲

حل مسائل Linked List

هفته ۳

حل مسائل Queue و Stack

هفته ۴

پیاده‌سازی ساختمان داده‌ها

هفته ۵

حل مسائل Hashing و Hash Table

هفته ۶

مدیریت پروژه

هفته ۷

حل مسائل Tree و BST

هفته ۸

پیاده‌سازی ساختمان داده‌ها

هفته ۹

حل مسائل Priority Queue و Heap

هفته ۱۰

حل مسائل ترکیبی

هفته ۱۱

حل مسائل Sorting

هفته ۱۲

حل مسائل جامع و آمادگی پروژه

هفته ۱۳

جمع‌بندی و آمادگی نهایی

هفته ۱۴

ساختار تمرین‌ها و ارزیابی بخش عملی

برنامه زمان‌بندی تمرین‌ها و پروژه

بخشی از ۲ نمره	مبانی الگوریتم، آرایه و رشته	سری تمرین ۱ ۳ مهر تا ۱۷ مهر
بخشی از ۲ نمره	روابط بازگشتی	سری تمرین ۲ ۱۷ مهر تا ۳ آبان
بخشی از ۲ نمره	لیست، پشته و صف	سری تمرین ۳ ۳ آبان تا ۱۵ آبان
بخشی از ۲ نمره	درخت‌ها و درهم‌سازی	سری تمرین ۴ ۱۵ آبان تا ۲۹ آبان
بخشی از ۲ نمره	مرتب‌سازی و مباحث پیشرفته	سری تمرین ۵ ۱۵ آذر تا ۲۹ آذر
۲ نمره	پیاپی‌سازی جامع پروژه	پروژه پایانی ۳۰ آذر

ارزیابی بخش عملی

۴

مجموع نمره
از ۲۰ نمره درس

۲

۵ سری تمرین عملی

پیاپی‌سازی و حل مسئله

۲

پروژه پایانی

پیاپی‌سازی جامع

قوانین ارسال، ارزیابی و احراز تسلط

۱ ارسال تمرین‌ها

- تمامی تمرین‌های عملی باید از طریق سامانه **Quera** ارسال شوند.
- نتیجه اجرای کد و داوری اولیه در همان بخش در لحظه انجام می‌شود.

۲ داوری و بررسی

- کدهای ارسالی پس از داوری خودکار، مجدداً بررسی می‌شوند.
- رعایت اصول آموزشی و صحت روند حل مسئله بررسی می‌شود.

۳ احراز تسلط

- امکان برگزاری ارائه یا توضیح شفاهی برای تمرین‌ها وجود دارد.
- هدف، اطمینان از تسلط کامل دانشجو بر راه‌حل ارائه‌شده است.

«استفاده از هر یک از زبان‌های برنامه نویسی شی‌گرا برای تمرینات و پروژه مجاز است.»

متغیرها، مدیریت حافظه، اشاره‌گرها و آرایه‌ها

داده‌ساختارها و الگوریتم‌ها | بخش عملی

متغیرها (Variables)

ذخیره‌سازی داده‌ها در خانه‌های حافظه

val

مدیریت حافظه (Memory Management)

مدیریت حافظه‌ی برنامه‌ها

val

اشاره‌گرها (Pointers)

ارجاع مستقیم به آدرس‌های حافظه

*ptr

آرایه‌ها (Arrays)

مجموعه‌های پیوسته از عناصر هم‌نوع

[0..n]

انواع داده (Data Types)

نوع داده مشخص می‌کند داده از چه دسته‌ای است و چه ویژگی‌هایی دارد؛ مانند اندازه و عملیات مجاز.

۱ انواع پایه (Built-in Types)

انواع ابتدایی داده که توسط زبان برنامه‌نویسی تعریف شده‌اند.

۲ آرایه‌ها (Arrays)

مجموعه‌ای از داده‌ها با نوع یکسان.

۳ ساختار و کلاس (Struct & Class)

مجموعه‌ای از داده‌ها و در صورت نیاز متدها، با امکان استفاده از انواع متفاوت.

۴ ساختمان داده‌ها (Data Structures)

مجموعه‌ای تعریف‌شده برای سازمان‌دهی داده‌ها و انجام عملیات روی داده‌ها.

مدیریت حافظه (Memory Management)

«سیستم عامل مسئول تخصیص، مدیریت و آزادسازی حافظه بین پروسه های در حال اجراست»

بخش های مختلف یک Process:

دستورالعمل های اجرا شونده برنامه

Code

متغیرهای عمومی و ایستا (Global/Static)

Data

تخصیص پویا (Dynamic Memory)

Heap

متغیرهای محلی و فراخوانی توابع

Stack

مکانیزم های مدیریت حافظه مجازی:

• Segmentation (بخش بندی)

تقسیم حافظه به بخش های منطقی متغیر

بخش های متغیر

• Paging (صفحه بندی)

تقسیم حافظه به بلاک های ثابت (Pages)

بلاک های ثابت

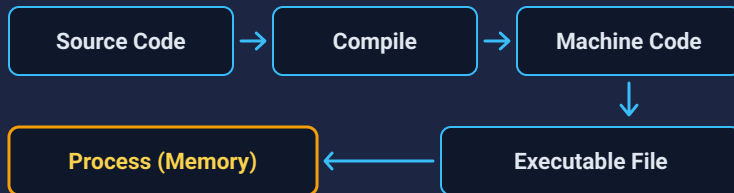
مدیریت حافظه: Segmentation

«در Segmentation، حافظه‌ی یک Process بر اساس بخش‌های منطقی آن به Segments های مختلف تقسیم می‌شود.»

نمونه کد ++C:

```
int main() {
    int x;
    char c;
    cin >> c;
    cout << x << c;
    return 0;
}
```

روند تبدیل کد به Process:



بخش‌های اصلی Process Segments:

Code

Code Segment

دستورالعمل‌های کامپایل‌شده برنامه (مانند main و دستورات cin/cout)

Data

Data Segment

متغیرهای عمومی (Global) و ایستا (Static)

Heap

Heap Segment

حافظه پویا (Dynamic Memory) تخصیص داده‌شده با new / malloc

Stack

Stack Segment

متغیرهای محلی (مانند x و c در main) و فریم فراخوانی توابع

Segmentation در حافظه

«Segment ها اندازه‌ی ثابتی ندارند و بر اساس نیاز Process شکل می‌گیرند.»

Process A

Code (32 KB)

Data (16 KB)

Heap (64 KB)

Stack (40 KB)

Process B

Code (16 KB)

Data (48 KB)

Heap (24 KB)

Stack (60 KB)

Process C

Code (56 KB)

Data (24 KB)

Heap (48 KB)

Stack (20 KB)

Process A

Free Space

Process B

Free Space

Process C

External Fragmentation: فضاهای خالی پراکنده در حافظه که به دلیل عدم هم‌اندازه بودن Segment ها ایجاد می‌شوند.

مدیریت حافظه: Paging

«Page ها اندازه‌ی ثابت دارند و می‌توانند در Frame های مختلف حافظه قرار بگیرند.»

Logical / Virtual Memory (Process A):

Page = بخش منطقی حافظه

Page 1

Page 2

Page 3

Page 4

Page 5

Physical Memory (RAM):

Frame = بخش فیزیکی حافظه

Frame 1 (Page 3)

Frame 2

Frame 3 (Page 1)

Frame 4

Frame 5

Frame 6 (Page 4)

Frame 7

Frame 8 (Page 2)

متغیرها (Variables)

• یک متغیر، جایی که در حافظه (Memory) برای ذخیره سازی داده است.

ویژگی های اصلی متغیر:

► تعداد بایت های تخصیص یافته (اندازه متغیر)

► محل قرارگیری حافظه (Address / آدرس)

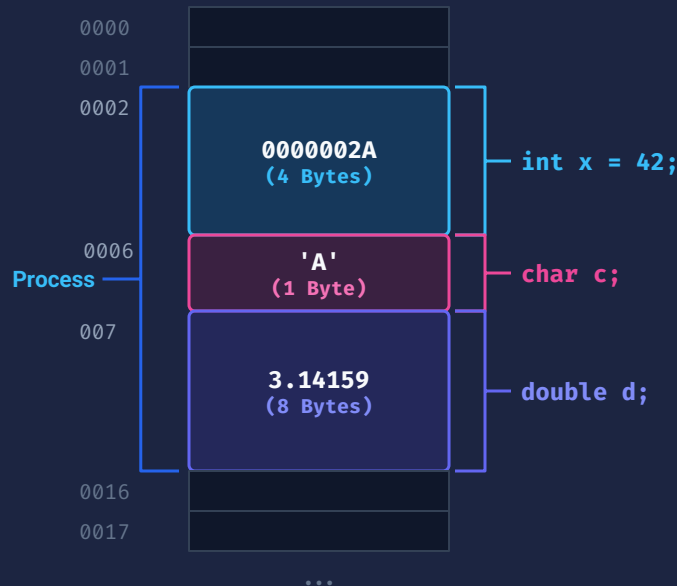
► عملیات تعریف شده / مجاز روی متغیر

• تبدیل نوع (Type Casting):

تبدیل یک نوع داده به نوع دیگر برای انجام عملیات یا ذخیره سازی مناسب.

```
int x = (int) 3.14; // Explicit Casting
```

نمودار حافظه (Memory Architecture)



اشاره‌گرها (Pointers)

- یک اشاره‌گر (Pointer)، متغیری است که برای ذخیره‌سازی آدرس حافظه استفاده می‌شود.
- از اشاره‌گر می‌توان برای دسترسی به داده‌ی آن آدرس استفاده کرد.

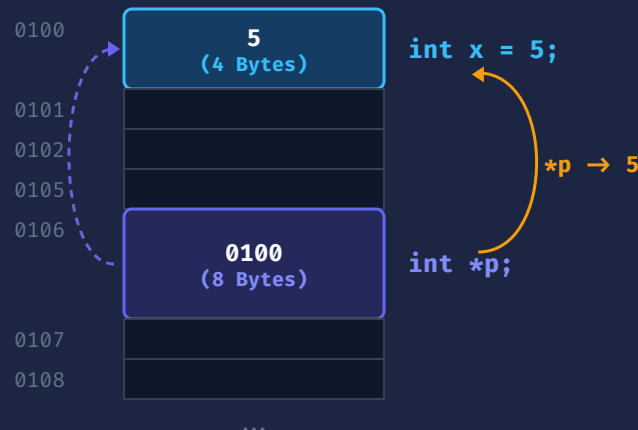
مثال C++:

```
int x = 5;
int *p;
p = &x;
cout << x << *p << &p;
```

• تحلیل مفاهیم در حافظه:

$x = 5$: مقدار متغیر برابر ۵ است.
 $p = \&x$: نگه داشتن آدرس x .
 $*p = 5$: خواندن مقدار موجود در آدرس.
 $\&x = 0100$ | $\&p = 0106$: آدرس‌های حافظه اختصاصی.

نمودار حافظه (Memory Architecture)



```
&x = 0100 (Address of x)
&p = 0106 (Address of pointer p)
p = 0100 (Value stored in p)
```

آرایه‌ها (Arrays)

- ناحیه‌ای پیوسته (Contiguous) از حافظه شامل عناصر هم‌اندازه.
- دسترسی تصادفی (Random Access): دسترسی سریع به هر عنصر با زمان ثابت.
- اندازه ثابت (Fixed Size): مشخص بودن اندازه حافظه از قبل.
- تخصیص بایت‌های متوالی در حافظه.

مثال ++C و نحوه اشاره به حافظه:

```
int x[3] = {1, 5, 7};
int *p = &x[0];
p++;
```

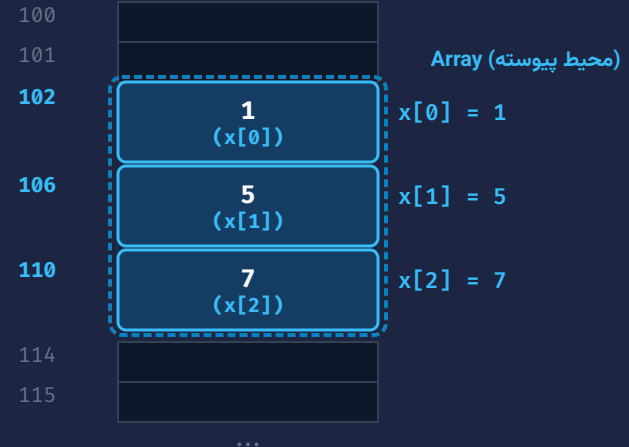
• محاسبه آدرس در:

`p++: p = p + sizeof(type)`

• زمان‌های مشخص شدن تعداد عناصر:

- **Compile Time** (Static Allocation) تخصیص ایستای حافظه
- **Run Time** (Dynamic Allocation) تخصیص پویای حافظه

نمودار چیدمان حافظه (Memory Layout)

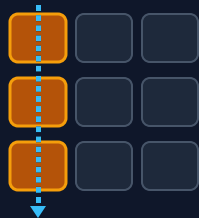


1. **Contiguous:** تمام عناصر آرایه پشت‌سرهم در حافظه قرار دارند
2. **Equal Size:** تمام عناصر هم‌نوع و هم‌اندازه‌اند
3. **Direct Access:** $\text{Base} + i * \text{Size} \rightarrow \text{O}(1)$

یک سوال برای تحلیل: جمع عناصر ماتریس

روش دوم: پیمایش ستونی

```
int sum = 0;
for (int j = 0; j < m; j++)
    for (int i = 0; i < n; i++)
        sum += a[i][j];
```



پیمایش ستون به ستون
(Column-Major)

روش اول: پیمایش سطری

```
int sum = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        sum += a[i][j];
```



پیمایش سطر به سطر
(Row-Major)

• سؤال:

این دو الگوریتم را تحلیل و با یکدیگر مقایسه کنید. آیا از نظر زمان اجرا تفاوتی دارند؟ آیا یکی از آن‌ها از دیگری بهتر است؟ دلیل خود را بیان کنید.

آرایه‌های چند بعدی و نحوه ذخیره‌سازی در حافظه

آرایه دوبعدی A (ماتریس 4×3 - دید منطقی)

A[0][0]	A[0][1]	A[0][2]	A[0][3]
A[1][0]	A[1][1]	A[1][2]	A[1][3]
A[2][0]	A[2][1]	A[2][2]	A[2][3]

$m = 3$ (سطرها)
 $n = 4$ (ستون‌ها)

روش‌های اصلی ذخیره‌سازی در حافظه:

Row-Major Order (سطر به سطر)

Column-Major Order (ستون به ستون)

چیدمان خطی در حافظه (Physical Memory)

A[0][0]

A[0][1]

A[0][2]

A[0][3]

A[1][0]

A[1][1]

...

A[2][3]

سطر اول (Row 0)

عناصر به صورت پیوسته و پشت سر هم در آدرس‌های حافظه قرار می‌گیرند.

محاسبه آدرس عنصر در ذخیره‌سازی سطر به سطر:

$$\text{Loc}(A[i][j]) = \text{Base}(A) + (i \times n + j) \times L$$

آرایه دوبعدی از نظر منطقی دارای سطر و ستون است، اما در حافظه فیزیکی رایانه به صورت یک فضای ۱D و خطی از عناصر متوالی ذخیره می‌شود.

برای بررسی بیشتر

تأثیر نحوه‌ی ذخیره‌سازی آرایه‌های دوبعدی در حافظه (Row / Column-Major Order) و مفهوم Cache Locality / Spatial Locality را حتماً بررسی کنید.

نحوه‌ی چیدمان در حافظه (Memory Layout)

Row-Major Order (C/C++)



Column-Major Order (Fortran/MATLAB)



مفهوم محلّیت داده‌ها (Cache Locality)

CPU Core

L1/L2 Cache

L3 Cache

Block
----->

Main Memory (RAM)

Cache Line (Fetched Data)

ساختار داده Struct / Class

- تعریف نوع داده جدید شامل انواع مختلف (Fieldها)
- یک نوع داده تعریف شده توسط کاربر (User-defined Type)

تعریف Struct و اندازه‌ی حافظه:



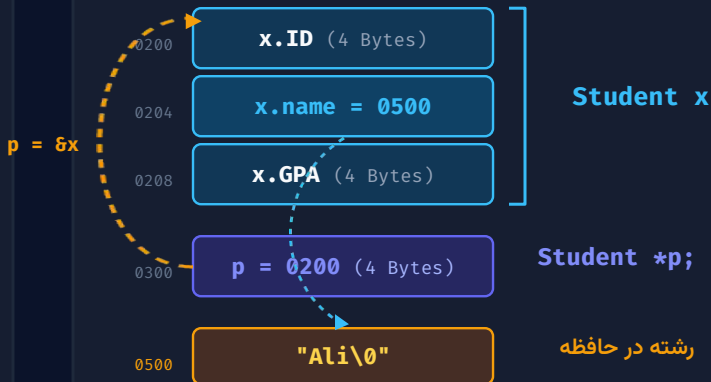
```
struct Student {
    int ID; // 4 bytes
    char* name; // 4 bytes (32-bit pointer)
    float GPA; // 4 bytes
};
```

• تخصیص متغیر و اشاره‌گر:

```
Student x;
Student *p = &x;
```

- ▶ متغیر **x** شامل ۱۲ بایت حافظه‌ی پیوسته است.
- ▶ اشاره‌گر **p** آدرس شروع متغیر **x** را نگه می‌دارد.

نمودار حافظه (Memory Architecture)



1. متغیر **x** اعضای Struct را به صورت یکپارچه در حافظه نگه می‌دارد.
2. اشاره‌گر **p** تنها آدرس بایت شروع **x** را ذخیره می‌کند.
3. عضو **name** خود یک اشاره‌گر است که به آدرس دیگری اشاره دارد.

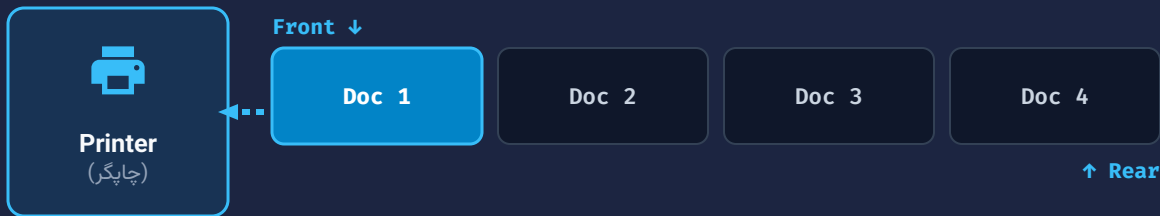
نوع داده انتزاعی (Abstract Data Type – ADT)

نوع داده انتزاعی (Abstract Data Type – ADT)

«ADT می‌گوید چه کاری انجام می‌شود، نه اینکه **چطور** انجام می‌شود.»

تعریف: کلاسی از اشیا که رفتار منطقی آن توسط مجموعه‌ای از مقادیر و مجموعه‌ای از عملیات تعریف می‌شود.

مثال: صف چاپگر (Printer Queue)



کاربر ۴ فایل برای چاپ ارسال می‌کند.
فایل‌ها در یک **Queue** قرار می‌گیرند و چاپگر آن‌ها را
به ترتیب پردازش می‌کند.

WHAT – رفتار صف (Interface)
`enqueue()` | `dequeue()` | `front()`
می‌دانیم چه عملیاتی روی صف انجام می‌شود.

HOW – پیاده‌سازی (Implementation)
Linked List یا **Array**
نحوه‌ی پیاده‌سازی برای استفاده‌کننده مهم نیست.

کاربر فقط با رفتار **Queue** کار می‌کند؛ نه با نحوه‌ی پیاده‌سازی آن.

دسته های از عملیات ها روی List - ADT



5. Destructor (تخریب‌کننده)

```
destroyList()  
free()
```

آزادسازی منابع، پاک‌سازی و نابودی فهرست.



4. Iterator (پیمایش‌گر)

```
first → next → next →  
...
```

دسترسی و حرکت ترتیبی روی تمام عناصر.



3. Observer (مشاهده‌گر)

```
size() | get(i)  
contains(id)
```

خواندن اطلاعات بدون ایجاد هیچ تغییری در فهرست.



2. Transformer (تغییردهنده)

```
add(student)  
remove(student)
```

محتوا و ساختار فهرست را تغییر می‌دهد.



1. Constructor (سازنده)

```
List<Student> s =  
createList();
```

ساخت و آماده‌سازی فهرست خالی دانشجویان.

الگوریتم‌ها (Algorithms)

الگوریتم‌ها (Algorithms)

مجموعه‌ای گام‌به‌گام و دقیق از دستورالعمل‌ها برای حل مسئله



۳. خروجی (Output)

نتیجه‌ی نهایی و مطلوب که پس از پایان محاسبات حاصل می‌شود.



۲. پردازش (Process)

مراحل منطقی، محاسباتی و معین که به ترتیب اجرا می‌شوند.



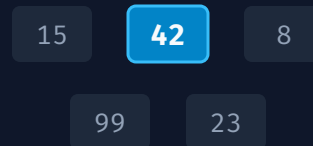
۱. ورودی (Input)

داده‌ها یا مقادیر اولیه که وارد الگوریتم می‌شوند تا پردازش روی آن‌ها انجام گیرد.

جست و جو در دنیای الگوریتم‌ها

«بسیاری از مسائل الگوریتمی، در اصل یک مسئله‌ی جست و جو هستند.»

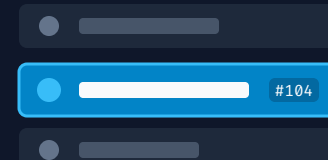
۱. پیدا کردن یک عدد



«آیا عدد ۴۲ در میان این داده‌ها وجود دارد؟»

جست‌وجوی یک مقدار خاص در مجموعه‌ای از داده‌های عددی.

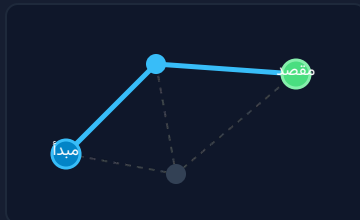
۲. پیدا کردن یک عنصر



«موقعیت یک دانشجو با شماره‌ی مشخص را در لیست پیدا کن.»

یافتن شیء یا رکورد خاص بر اساس کلید شناسه.

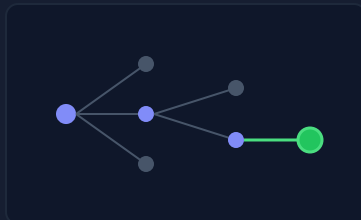
۳. کوتاه‌ترین مسیر



«کوتاه‌ترین مسیر بین دو نقطه را پیدا کن.»

جست‌وجو در گراف برای یافتن بهینه‌ترین مسیر ارتباطی.

۴. پیدا کردن راه‌حل



«در میان حالت‌های ممکن، حالتی را پیدا کن که شرط مسئله را برآورده کند.»

پیمایش فضای حالت برای کشف پاسخ مسئله.

چطور این جست‌وجو را سریع‌تر انجام دهیم؟

جست و جوی خطی (Linear Search)

«در جست و جوی خطی، عناصر یکی یکی و به ترتیب بررسی می شوند تا عنصر مورد نظر پیدا شود.»

مثال: پیدا کردن عدد 42 در آرایه



روش ترتیبی (Iterative)

«با یک حلقه، عناصر را یکی یکی بررسی می کنیم.»
استفاده از حلقه های تکرار مثل for یا while برای پیمایش آرایه.

روش بازگشتی (Recursive)

«هر بار یک عنصر بررسی می شود و ادامه ی جست و جو به فراخوانی بعدی واگذار می شود.»
شکستن مسئله به زیرمسئله های کوچک تر با فراخوانی تابع.

Time Complexity: Best Case $\rightarrow O(1)$ | Worst Case $\rightarrow O(n)$

«در بدترین حالت، ممکن است مجبور شویم تمام عناصر را بررسی کنیم.»

داده‌های مرتب‌یافته (Sorted)

جستجوی دودویی (Binary Search) - کاهش ۵۰ درصدی فضای جستجو در هر گام

```

BS(A, L, R, t):
    if L > R:
        return -1
    mid = L + (R - L) / 2
    if A[mid] == t:
        return mid
    else if A[mid] < t:
        return BS(A, mid + 1, R, t)
    else:
        return BS(A, L, mid - 1, t)
  
```

```

BS(A, n, t):
    L = 0
    R = n - 1
    while (L <= R):
        mid = L + (R - L) / 2
        if A[mid] == t:
            return mid
        else if A[mid] < t:
            L = mid + 1
        else:
            R = mid - 1
    return -1
  
```

روش ترتیبی (Iterative)

- استفاده از حلقه تکرار
- کنترل محدوده با اشاره‌گرهای
- بدون مصرف حافظه اضافی پشته (Stack Overhead)

روش بازگشتی (Recursive)

- فایده: پیدا کردن مقدار یا خالی شدن بازه
- کد تمیزتر با منطق تقسیم و غلبه (Divide & Conquer)
- $\text{return } -1$

Time Complexity: $O(\log_2 n)$ | Space Complexity: Iterative $\rightarrow O(1)$ | Recursive $\rightarrow O(\log n)$



ممنون از حضورتون

امیدوارم این جلسه براتون مفید و روشن بوده باشه

جلسه بعد

میریما سراغ حل مسائل

از نحوه‌ی فکر کردن و تحلیل صورت سؤال تا انتخاب الگوریتم و پیاده‌سازی

تا جلسه بعد ✨